

Makeup Assignment(s): Single-Cycle Processor Design

Instructions Implemented: add and xor

Armando Galvan

4/15/2026

CSC4210 Computer Architecture

Instructor: Professor Mohammed Alser, PhD

Why I chose add and xor

- I decided on these because they have similar formats since they are both R type instructions.
- Benefit of choosing both of these are that since they are both R type instructions, that also means they have similar datapaths. So I can make a processor design and reuse it for the other operation.

R-Type Instruction Format

add rd, rs1, rs2

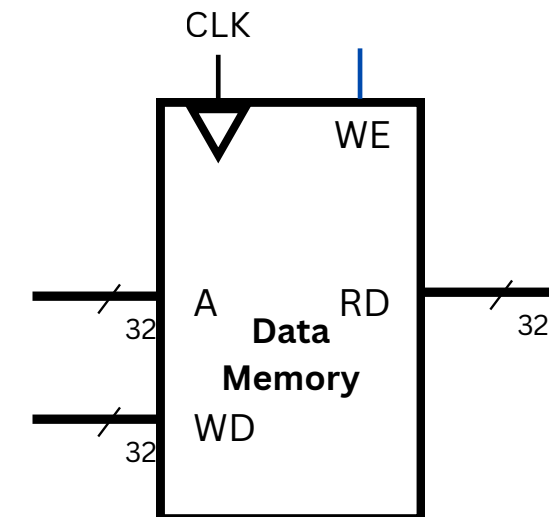
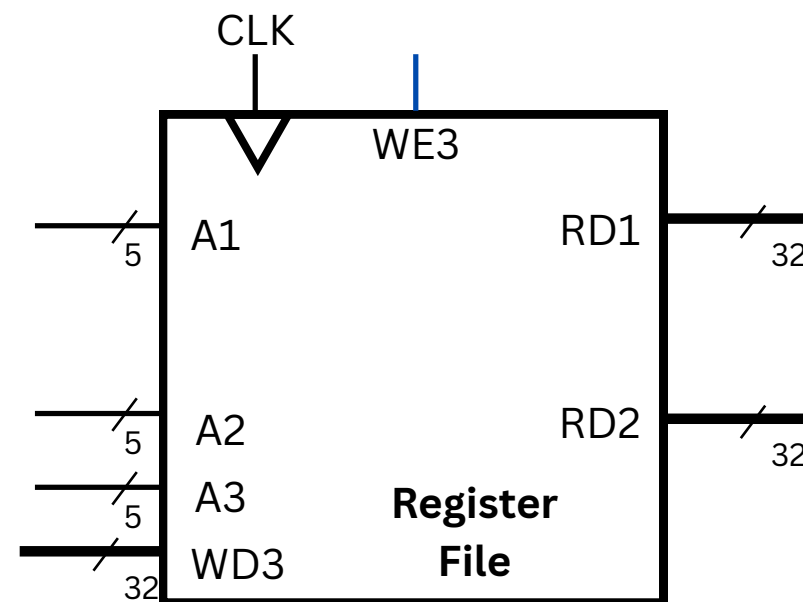
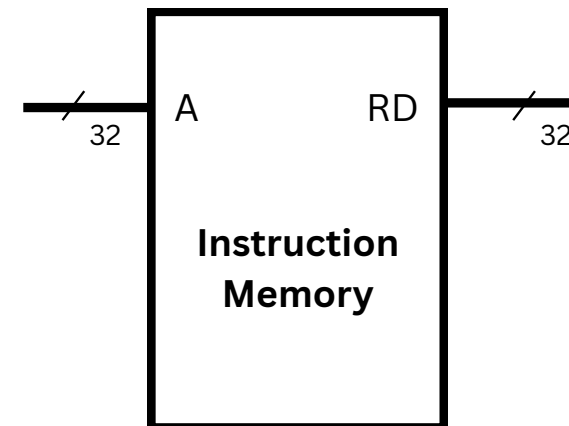
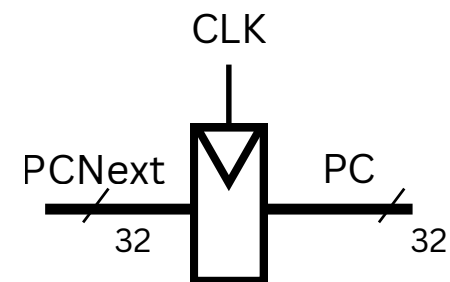
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	-	-	-	-	-	-	-	-	-	-	0	0	0	-	-	-	-	-	0	1	1	0	0	1	1
funct7							rs2					rs1					funct3			rd					opcode						

xor rd, rs1, rs2

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	-	-	-	-	-	-	-	-	-	-	1	0	0	-	-	-	-	-	0	1	1	0	0	1	1
funct7							rs2					rs1					funct3			rd					opcode						

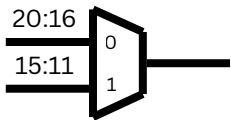
Main Components

I started by drawing out main datapath components. Those being Program Counter, Instruction Memory, Register File and Data Memory.

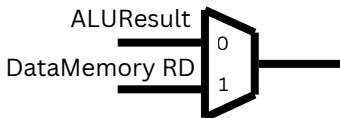


Additional Components

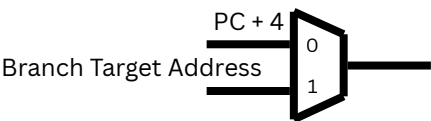
RegDst MUX



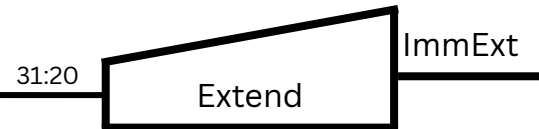
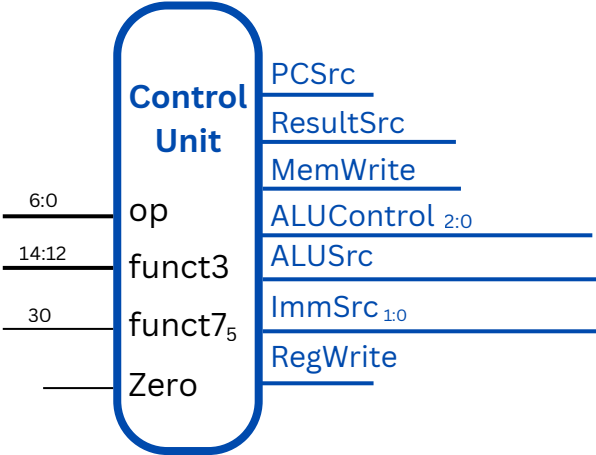
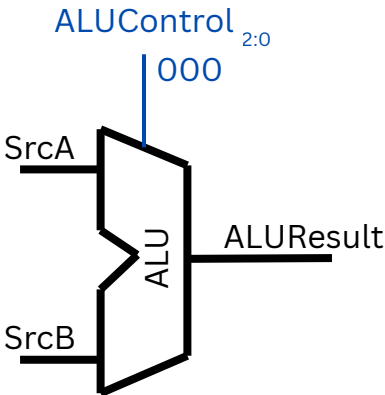
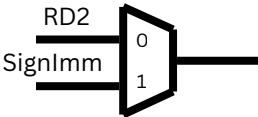
MemtoReg
MUX



PCSrc MUX

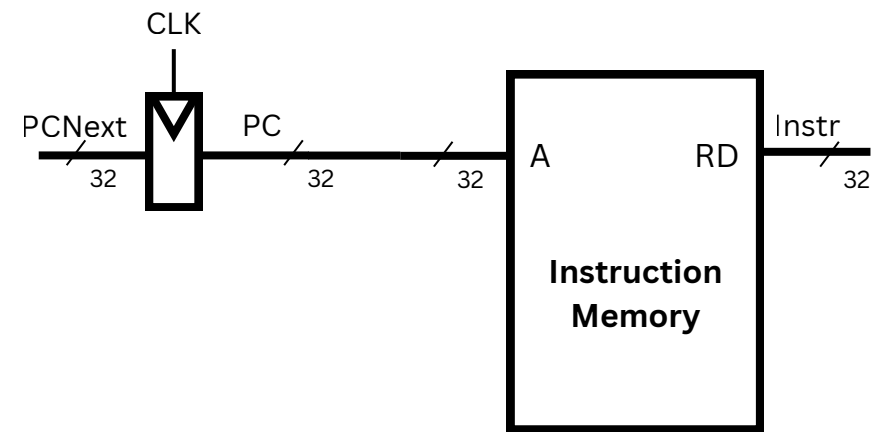


ALUSrc MUX

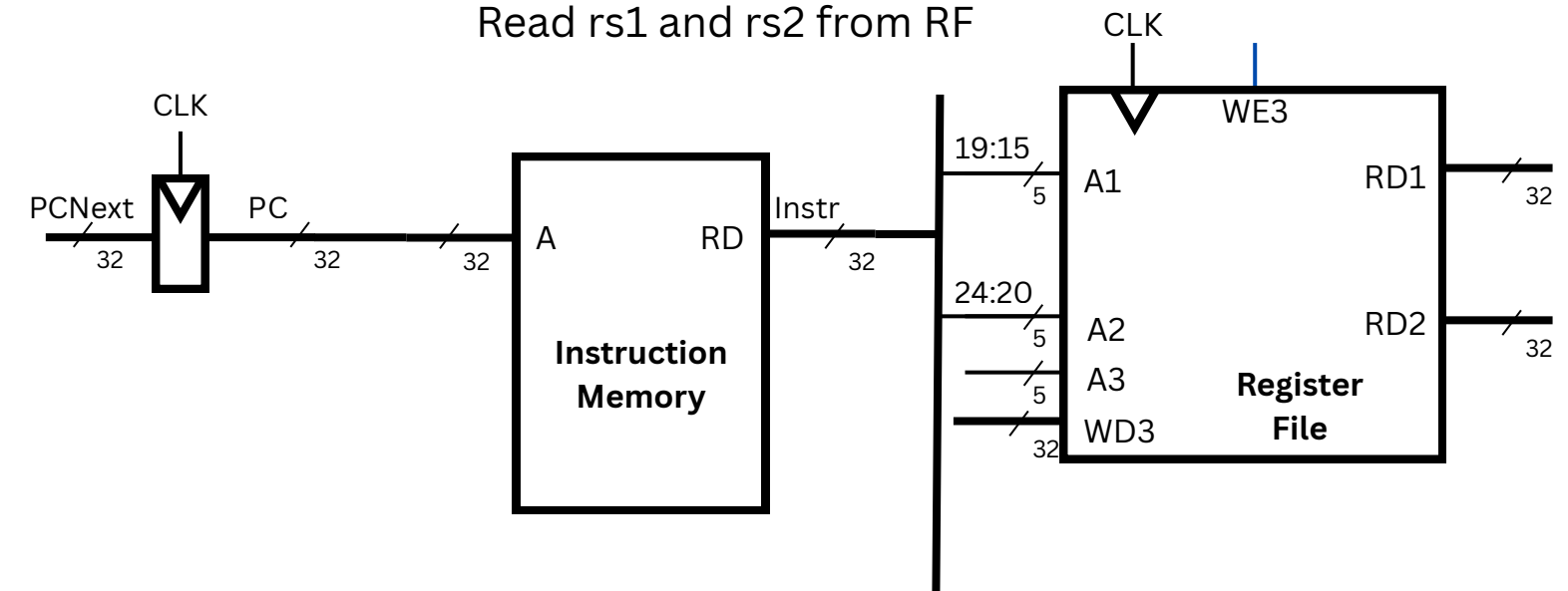


Implementation

Fetch Instruction

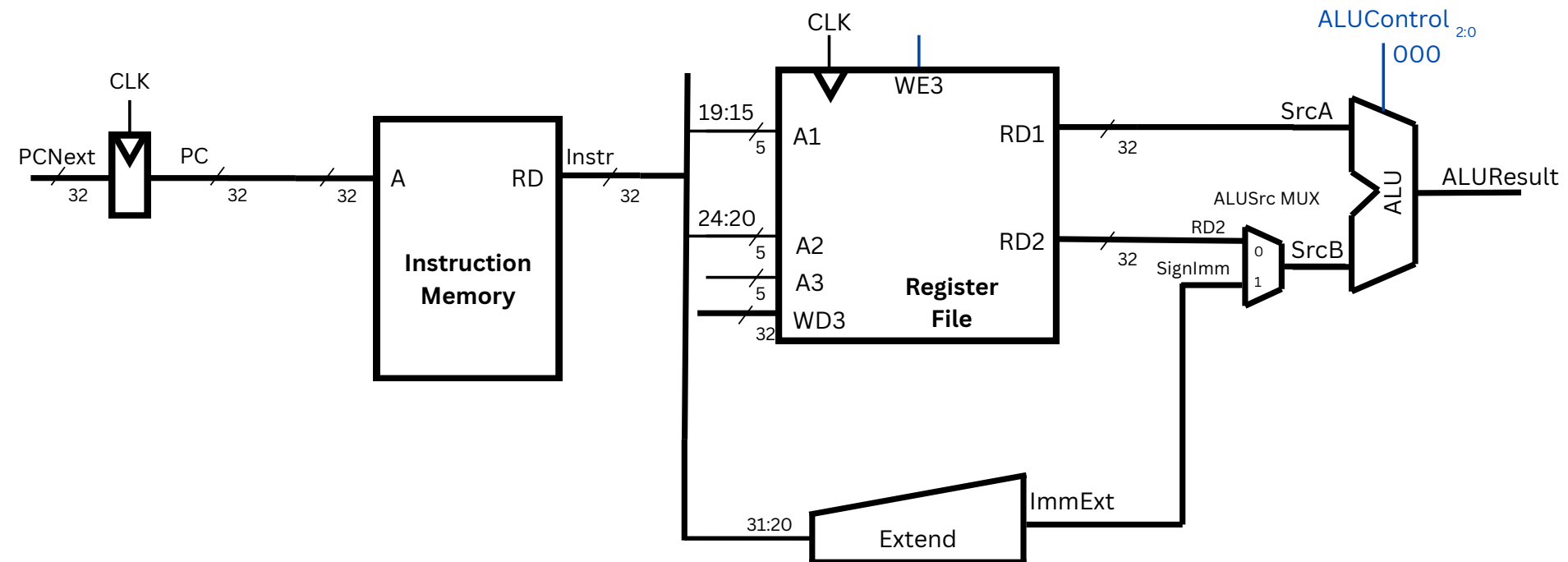


Read rs1 and rs2 from RF



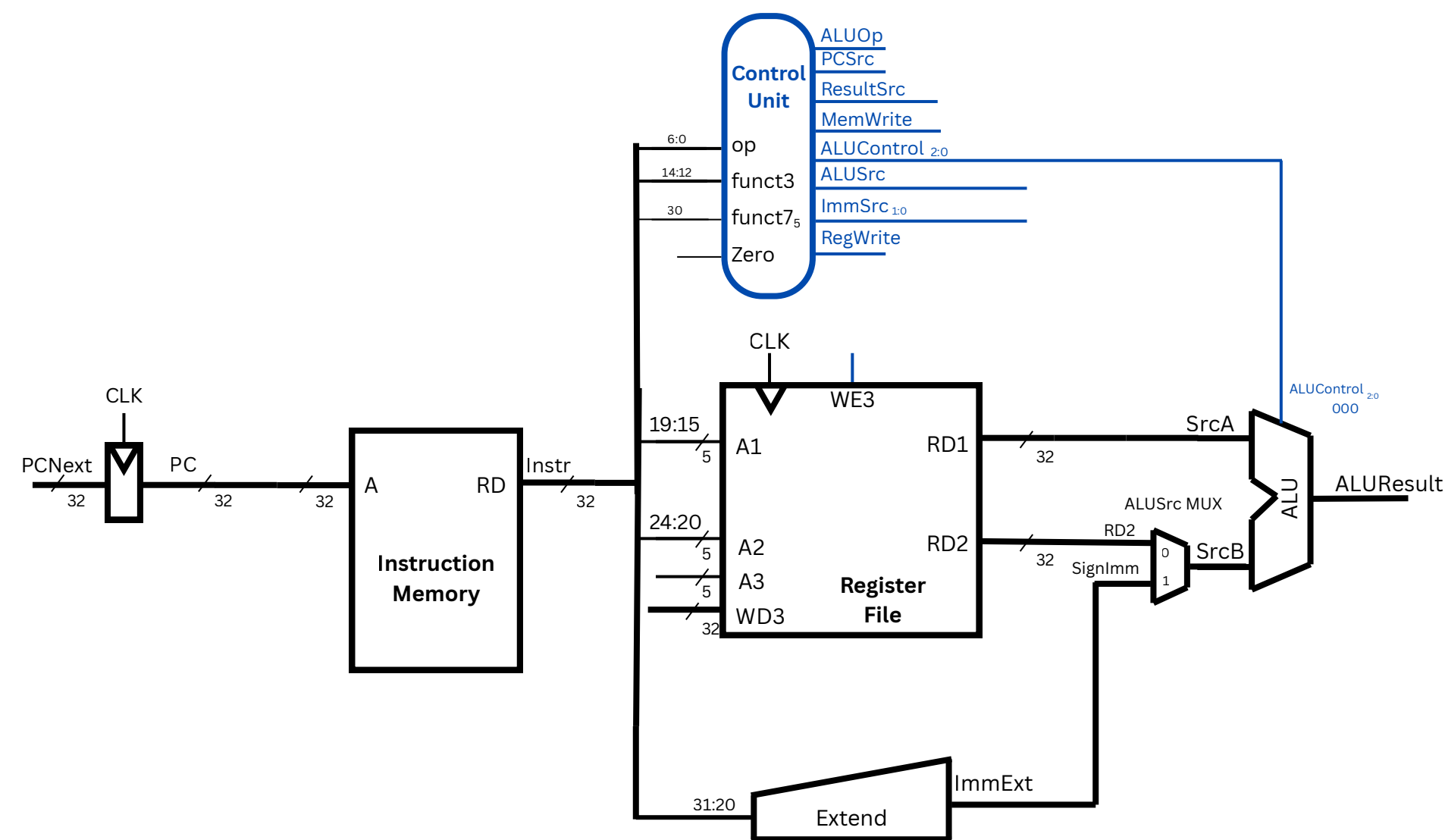
Implementation Continued

Connect RD1, and RD2 to ALU. Because this is R type, ALUSrc Mux will always pass 0 (the register value)



Implementation Continued

Pass on funct3 and funct7 from Instruction Memory to ALU control unit. To avoid drawing later I'll go ahead and connect opcode.

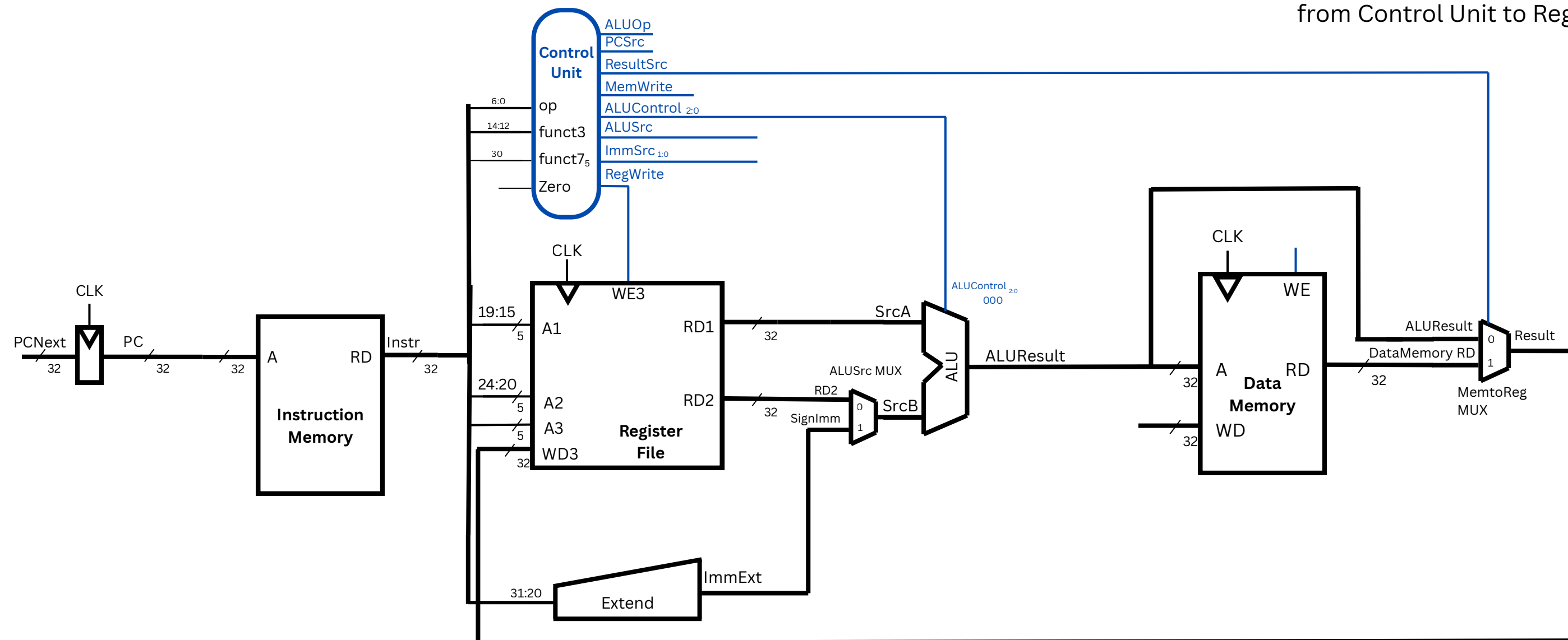


Connect ALUControl to ALU Control Unit

Implementation Continued

Connect ALU Result to Data Memory and MemtoReg Mux.
Connect RD to MemtoReg, and connect ResultSrc to MemtoReg.
The connect result to Register File WD3, to deliver value to be written into destination register.

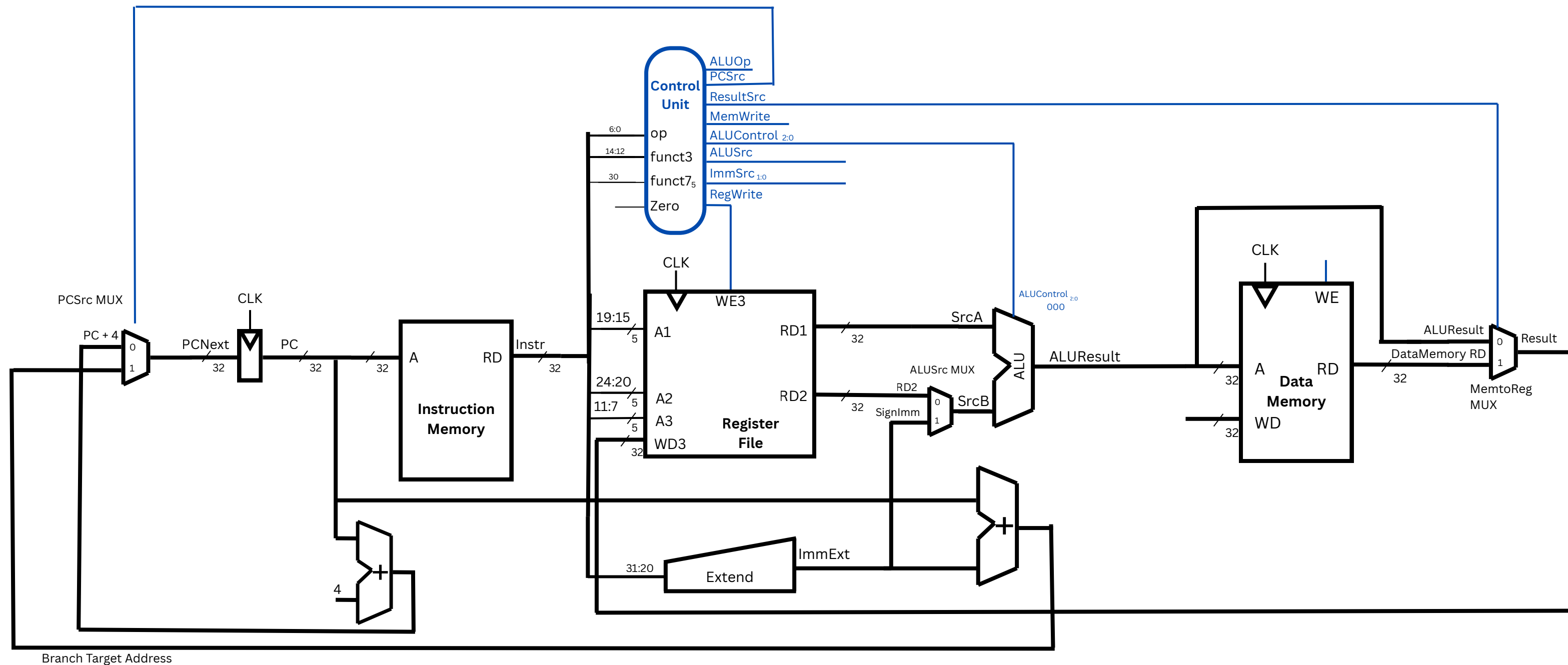
Make sure Instruction Memory connects to Register File A3 so the register file knows which register to write to. And that Write is enable from Control Unit to Register File WE3.



Implementation Continued

After all that the only instruction left is increment PC by 4.
So we add an added to computer PC + 4. We will also include a PCSrc Mux input 1 based on the slide power point 17, event though it won't be used for operations Xor and add.

We also have ot make sure PCSrc is connect to PCSrc Mux. And thats it.



Summary Table for add and xor

Stage	Components	What happens
Fetch	PC → Instruction Memory	Get instruction
Decode	Instruction Memory → Control Unit, Register File, Sign Extend	Decode and read registers
Execute	ALUSrc MUX → ALU	Perform add or xor
Memory	ALU → Data Memory	Skipped (no memory access)
Write Back	MemtoReg MUX → Register File	Write ALU result to rd
PC Update	PC+4 → PCSrc MUX → PC	Update to next instruction